

Introduction to Python Programming

Chapter 8: Lists - Intermediate

Introduction

Now that you understand the basics of lists, it's time to explore more powerful operations. This chapter covers how to modify lists, understand what happens in memory when you work with lists, and use advanced features like slicing. These concepts are crucial for writing effective Python programs.

1. Understanding Mutability

Mutability means something can be changed after it's created. Lists are **mutable** - you can modify them. Strings are **immutable** - you cannot change them.

Mutable: Lists

```
# Lists CAN be changed
fruits = ["apple", "banana", "cherry"]
fruits[1] = "blueberry" # This works!
print(fruits) # ['apple', 'blueberry', 'cherry']
```

Immutable: Strings

```
# Strings CANNOT be changed
word = "hello"
word[0] = "H" # ERROR! Strings are immutable
```

To "change" a string, you must create a new one:

```
word = "hello"
word = "H" + word[1:] # Create a new string
print(word) # "Hello"
```

Why Does Mutability Matter?

Mutability affects how Python handles variables and memory. This becomes critical when copying lists or passing them to functions (which we'll learn about later).

2. References vs Copies: The Critical Concept

This is one of the most important (and tricky) concepts in Python lists.

Assignment Creates a Reference, Not a Copy

```
list_a = [1, 2, 3]
list_b = list_a # This does NOT create a copy!

list_b[0] = 99

print(list_a) # [99, 2, 3] - CHANGED!
print(list_b) # [99, 2, 3]
```

What happened? Both `list_a` and `list_b` point to the SAME list in memory. Changing one changes the other!

Creating an Actual Copy

To create a separate, independent list, use `.copy()`:

```
list_a = [1, 2, 3]
list_b = list_a.copy() # This DOES create a copy!

list_b[0] = 99

print(list_a) # [1, 2, 3] - UNCHANGED
print(list_b) # [99, 2, 3]
```

Now they're separate lists. Changes to one don't affect the other.

Visualizing the Difference

Assignment (reference):

```
list_a → [1, 2, 3]
      ↑
list_b →
```

Both variables point to the same list.

Copy:

```
list_a → [1, 2, 3]

list_b → [1, 2, 3] (separate copy)
```

Each variable points to its own list.

When to Use Each

Use assignment (`list_b = list_a`) when:

- You want two names for the same list
- Changes should affect both variables
- You want to save memory (only one list exists)

Use copy (`list_b = list_a.copy()`) when:

- You want independent lists
- Changes should NOT affect the original
- You need to preserve the original while modifying a copy

For more detail on why this happens, see Appendix A.

3. Adding Elements to Lists

Append: Add to the End

The most common way to add an item:

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)  # ['apple', 'banana', 'cherry']
```

Insert: Add at a Specific Position

```
fruits = ["apple", "cherry"]
fruits.insert(1, "banana")  # Insert at index 1
print(fruits)  # ['apple', 'banana', 'cherry']
```

Format: `list.insert(index, item)`

Extend: Add Multiple Items

```
fruits = ["apple", "banana"]
more_fruits = ["cherry", "date"]
fruits.extend(more_fruits)
print(fruits)  # ['apple', 'banana', 'cherry', 'date']
```

Concatenation: Combine Lists with +

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2
print(combined) # [1, 2, 3, 4, 5, 6]
```

Note: This creates a NEW list. The originals are unchanged.

Building a List Dynamically

```
numbers = []

for i in range(5):
    numbers.append(i * 2)

print(numbers) # [0, 2, 4, 6, 8]
```

4. Removing Elements from Lists

Remove: Delete by Value

Removes the FIRST occurrence of the value:

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
print(fruits) # ['apple', 'cherry', 'banana']
```

Warning: If the value doesn't exist, you get an error:

```
fruits = ["apple", "cherry"]
fruits.remove("banana") # ERROR! ValueError
```

Pop: Remove by Index (and Return It)

```
fruits = ["apple", "banana", "cherry"]
removed = fruits.pop(1) # Remove index 1
print(removed) # banana
print(fruits) # ['apple', 'cherry']
```

Pop without an index removes the LAST item:

```
fruits = ["apple", "banana", "cherry"]
last = fruits.pop()
print(last)    # cherry
print(fruits)  # ['apple', 'banana']
```

Del: Delete by Index

```
fruits = ["apple", "banana", "cherry"]
del fruits[1]
print(fruits)  # ['apple', 'cherry']
```

You can also delete entire slices (we'll learn slicing soon):

```
numbers = [1, 2, 3, 4, 5]
del numbers[1:3]  # Delete indices 1 and 2
print(numbers)   # [1, 4, 5]
```

Clear: Empty the List

```
fruits = ["apple", "banana", "cherry"]
fruits.clear()
print(fruits)  # []
```

5. Modifying Elements

Simply assign a new value to an index:

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "blueberry"
print(fruits)  # ['apple', 'blueberry', 'cherry']
```

You can modify based on the current value:

```
numbers = [10, 20, 30]
numbers[0] = numbers[0] * 2
print(numbers)  # [20, 20, 30]
```

Or modify all elements:

```
numbers = [1, 2, 3, 4, 5]

for i in range(len(numbers)):
    numbers[i] = numbers[i] * 10

print(numbers) # [10, 20, 30, 40, 50]
```

6. Finding Items in Lists

The Old Way: Manual Search

Before we knew about built-in tools:

```
fruits = ["apple", "banana", "cherry", "date"]
search_for = "cherry"
found = False
position = -1

for i in range(len(fruits)):
    if fruits[i] == search_for:
        found = True
        position = i
        break

if found:
    print(f"Found '{search_for}' at index {position}")
else:
    print(f"'{search_for}' not found")
```

This works, but there are better ways!

The `in` Operator: Check if Item Exists

```
fruits = ["apple", "banana", "cherry"]

if "banana" in fruits:
    print("Yes, banana is in the list")

if "grape" not in fruits:
    print("No, grape is not in the list")
```

Much cleaner!

The `index()` Method: Find Position

```
fruits = ["apple", "banana", "cherry", "date"]
position = fruits.index("cherry")
print(position) # 2
```

Warning: If the item doesn't exist, you get an error:

```
fruits = ["apple", "banana", "cherry"]
position = fruits.index("grape") # ERROR! ValueError
```

Better approach - check first:

```
search_for = "grape"

if search_for in fruits:
    position = fruits.index(search_for)
    print(f"Found at index {position}")
else:
    print("Not found")
```

The `count()` Method: How Many Times?

```
numbers = [1, 2, 3, 2, 4, 2, 5]
count = numbers.count(2)
print(count) # 3
```

7. List Slicing

Slicing lets you get a portion of a list. This is incredibly powerful!

Basic Slicing Syntax

Format: `list[start:stop]`

- Includes `start` index
- Excludes `stop` index (just like range!)

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5]) # [2, 3, 4]
print(numbers[0:3]) # [0, 1, 2]
print(numbers[5:8]) # [5, 6, 7]
```

Omitting Start or Stop

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[:5])    # [0, 1, 2, 3, 4] (start from beginning)
print(numbers[5:])    # [5, 6, 7, 8, 9] (go to end)
print(numbers[:])     # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] (entire list)
```

Negative Indices in Slicing

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[-3:])   # [7, 8, 9] (last 3 items)
print(numbers[:-2])   # [0, 1, 2, 3, 4, 5, 6, 7] (all but last 2)
print(numbers[-5:-2]) # [5, 6, 7]
```

Step in Slicing

Format: `list[start:stop:step]`

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[:2])    # [0, 2, 4, 6, 8] (every 2nd item)
print(numbers[1::2])  # [1, 3, 5, 7, 9] (every 2nd, starting at 1)
print(numbers[::3])   # [0, 3, 6, 9] (every 3rd item)
```

Reversing with Slicing

```
numbers = [1, 2, 3, 4, 5]
reversed_numbers = numbers[::-1]
print(reversed_numbers) # [5, 4, 3, 2, 1]
```

The `-1` step means "go backwards"!

Practical Slicing Examples

```
# Get first 3 items
scores = [85, 92, 78, 90, 88, 76]
top_3 = scores[:3]
print(top_3) # [85, 92, 78]

# Get middle items
middle = scores[2:-2]
print(middle) # [78, 90]

# Copy a list using slicing
original = [1, 2, 3, 4, 5]
copy = original[:] # Same as original.copy()
```

8. String Slicing

Everything we learned about list slicing works on strings too!

Basic String Slicing

```
text = "Python Programming"

print(text[0:6])    # Python
print(text[7:])     # Programming
print(text[:6])     # Python
print(text[-11:])   # Programming
```

Extracting Substrings

```
email = "user@example.com"

# Get username (before @)
at_position = email.index("@")
username = email[:at_position]
print(username) # user

# Get domain (after @)
domain = email[at_position + 1:]
print(domain) # example.com
```

String Manipulation with Slicing

```
# Reverse a string
word = "Python"
reversed_word = word[::-1]
print(reversed_word) # nohtyP

# Every other character
text = "abcdefgh"
print(text[::2]) # aceg

# First and last 3 characters
sentence = "Hello World"
print(sentence[:3]) # Hel
print(sentence[-3:]) # rld
```

9. Sorting and Reversing Lists

Sort: Modify the List

```
numbers = [5, 2, 8, 1, 9]
numbers.sort()
print(numbers) # [1, 2, 5, 8, 9]

# Sort in reverse
numbers.sort(reverse=True)
print(numbers) # [9, 8, 5, 2, 1]
```

Sorted: Create a New Sorted List

```
numbers = [5, 2, 8, 1, 9]
sorted_numbers = sorted(numbers)
print(numbers) # [5, 2, 8, 1, 9] (unchanged)
print(sorted_numbers) # [1, 2, 5, 8, 9]
```

Reverse: Flip the Order

```
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers) # [5, 4, 3, 2, 1]
```

Sorting Strings

```
words = ["banana", "apple", "cherry", "date"]
words.sort()
print(words)  # ['apple', 'banana', 'cherry', 'date']
```

10. Practical Examples

Example 1: Remove Duplicates

```
numbers = [1, 2, 3, 2, 4, 3, 5, 1]
unique = []

for number in numbers:
    if number not in unique:
        unique.append(number)

print(unique)  # [1, 2, 3, 4, 5]
```

Example 2: Split a List

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
midpoint = len(numbers) // 2

first_half = numbers[:midpoint]
second_half = numbers[midpoint:]

print(first_half)  # [1, 2, 3, 4, 5]
print(second_half)  # [6, 7, 8, 9, 10]
```

Example 3: Filter List

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = []

for number in numbers:
    if number % 2 == 0:
        even_numbers.append(number)

print(even_numbers)  # [2, 4, 6, 8, 10]
```

Example 4: List Statistics

```
scores = [85, 92, 78, 90, 88]
```

```
total = sum(scores)
average = total / len(scores)
maximum = max(scores)
minimum = min(scores)
```

```
print(f"Average: {average}")
print(f"Highest: {maximum}")
print(f"Lowest: {minimum}")
```

Example 5: Working with Names

```
full_name = "John Michael Smith"
names = full_name.split() # Split into list

first_name = names[0]
last_name = names[-1]
middle_name = names[1] if len(names) > 2 else ""

print(f"First: {first_name}")
print(f"Middle: {middle_name}")
print(f>Last: {last_name}")
```

Key Takeaways

- ✓ **Lists are mutable** - can be changed after creation
 - ✓ **Assignment creates reference** - `b = a` makes both point to same list
 - ✓ **Use `.copy()` for independent copy** - `b = a.copy()` creates separate list
 - ✓ **Add items:** `append()`, `insert()`, `extend()`, or `+` operator
 - ✓ **Remove items:** `remove()`, `pop()`, `del`, or `clear()`
 - ✓ **Find items:** `in` operator, `index()`, `count()`
 - ✓ **Slicing:** `[start:stop:step]` - stop is excluded
 - ✓ **String slicing:** Works exactly like list slicing
 - ✓ **Sort:** `sort()` modifies list, `sorted()` creates new one
 - ✓ **Reverse:** `reverse()` modifies list, `[::-1]` creates new one
-

Reflection Questions

1. What's the difference between mutable and immutable?
 2. What happens when you write `list_b = list_a`?
 3. How do you create an independent copy of a list?
 4. What's the difference between `remove()` and `pop()`?
 5. What does `list[2:5]` return?
 6. How do you get the last 3 items of a list using slicing?
 7. What does `[::-1]` do?
 8. What's the difference between `sort()` and `sorted()`?
-

Practice Exercises

1. **List Operations:** Create a list, add 5 items, remove 2, modify 1, print result
 2. **Copy Test:** Create a list, make a copy, modify the copy, verify original unchanged
 3. **Merge Lists:** Take two lists of numbers, combine them, remove duplicates, sort
 4. **Slice Practice:** Given a list of 10 items, extract first 3, last 3, middle 4
 5. **Name Parser:** Ask for full name, split into first/last, capitalize each
 6. **Shopping List:** Build a program that adds/removes items, shows current list
 7. **Grade Manager:** Store grades, calculate average, find highest/lowest
 8. **Palindrome Checker:** Use slicing to check if a string is a palindrome
 9. **List Filter:** Create a list of numbers, create new list with only values > 50
 10. **Word Reverser:** Take a sentence, reverse each word, print reversed sentence
-

Appendix A: References and Memory

What Really Happens with Assignment

When you create a list, Python stores it somewhere in memory. The variable name is just a "label" that points to that memory location.

Creating a list:

```
list_a = [1, 2, 3]
```

In memory:

```
Variable: list_a
  ↓
Memory Address: 1000
  ↓
Actual Data: [1, 2, 3]
```

Assignment Creates Another Label

When you write `list_b = list_a`:

```
list_a = [1, 2, 3]
list_b = list_a
```

In memory:

```
Variable: list_a → Memory Address: 1000 → [1, 2, 3]
Variable: list_b _____ ↑
```

Both variables point to the SAME memory location. There's only ONE list!

When you modify through either variable:

```
list_b[0] = 99
```

You're modifying the single list that both variables point to:

```
Variable: list_a → Memory Address: 1000 → [99, 2, 3]
Variable: list_b _____ ↑
```

Copy Creates New Memory

When you write `list_b = list_a.copy()`:

```
list_a = [1, 2, 3]
list_b = list_a.copy()
```

In memory:

```
Variable: list_a → Memory Address: 1000 → [1, 2, 3]
Variable: list_b → Memory Address: 2000 → [1, 2, 3]
```

Now there are TWO separate lists in TWO different memory locations.

Modifying one doesn't affect the other:

```
list_b[0] = 99
```

Variable: list_a → Memory Address: 1000 → [1, 2, 3] (unchanged)

Variable: list_b → Memory Address: 2000 → [99, 2, 3] (modified)

Why This Matters: Efficiency

Lists can be HUGE. Imagine a list with 1 million items.

Assignment (reference):

- Creates a new variable name
- Points to existing list
- Fast! No data copied
- Uses minimal memory

Copy:

- Creates new variable name
- Creates entirely new list
- Copies ALL 1 million items
- Slow for large lists
- Uses double the memory

Python defaults to references for efficiency. It assumes you want to save time and memory unless you explicitly ask for a copy.

When This Causes Problems

```
def process_list(data):  
    data.append(999) # Modifies the original!  
    return data  
  
my_list = [1, 2, 3]  
result = process_list(my_list)  
  
print(my_list) # [1, 2, 3, 999] - Original changed!
```

If you want to protect the original, pass a copy:

```
result = process_list(my_list.copy())  
print(my_list)  # [1, 2, 3] - Original safe!
```

The Bottom Line

- **Assignment (=)** creates a reference - both variables share one list
- **Copy (.copy())** creates a duplicate - each variable has its own list
- References are efficient but can cause unexpected changes
- Copies are safer but use more time and memory

Understanding this is crucial as you write more complex programs!